

After Step E:

Started by user N1419979

Obtained Jenkinsfile from git <http://gitea:3000/N1419979/lab9-advanced.git>

org.codehaus.groovy.control.MultipleCompilationErrorsException: startup failed:

WorkflowScript: 102: unexpected char: '\' @ line 102, column 47.

```
m = re.search(r"src/([\w.]+\.
```

```
py)
```

```
^
```

1 error

at org.codehaus.groovy.control.ErrorCollector.failIfErrors(ErrorCollector.java:309)

at org.codehaus.groovy.control.ErrorCollector.addFatalError(ErrorCollector.java:149)

at org.codehaus.groovy.control.ErrorCollector.addError(ErrorCollector.java:119)

at org.codehaus.groovy.control.ErrorCollector.addError(ErrorCollector.java:131)

at org.codehaus.groovy.control.SourceUnit.addError(SourceUnit.java:349)

at

org.codehaus.groovy.antlr.AntlrParserPlugin.transformCSTIntoAST(AntlrParserPlugin.java:220)

at org.codehaus.groovy.antlr.AntlrParserPlugin.parseCST(AntlrParserPlugin.java:191)

at org.codehaus.groovy.control.SourceUnit.parse(SourceUnit.java:233)

at org.codehaus.groovy.control.CompilationUnit\$1.call(CompilationUnit.java:189)

at

org.codehaus.groovy.control.CompilationUnit.applyToSourceUnits(CompilationUnit.java:966)

at

org.codehaus.groovy.control.CompilationUnit.doPhaseOperation(CompilationUnit.java:626)

at

org.codehaus.groovy.control.CompilationUnit.processPhaseOperations(CompilationUnit.java:602)

at org.codehaus.groovy.control.CompilationUnit.compile(CompilationUnit.java:579)

at groovy.lang.GroovyClassLoader.doParseClass(GroovyClassLoader.java:323)

at groovy.lang.GroovyClassLoader.parseClass(GroovyClassLoader.java:293)

at PluginClassLoader for

script-security//org.jenkinsci.plugins.scriptsecurity.sandbox.groovy.GroovySandbox\$Scope.parse(GroovySandbox.java:162)

```
at PluginClassLoader for
workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsGroovyShell.doParse(CpsGroovyS
hell.java:202)
at PluginClassLoader for
workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsGroovyShell.reparse(CpsGroovyS
hell.java:186)
at PluginClassLoader for
workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsFlowExecution.parseScript(CpsFl
owExecution.java:669)
at PluginClassLoader for
workflow-cps//org.jenkinsci.plugins.workflow.cps.CpsFlowExecution.start(CpsFlowExe
cution.java:615)
at PluginClassLoader for
workflow-job//org.jenkinsci.plugins.workflow.job.WorkflowRun.run(WorkflowRun.java:34
1)
at hudson.model.ResourceController.execute(ResourceController.java:97)
at hudson.model.Executor.run(Executor.java:456)
Finished: FAILURE
```

Part 4 Fix — Move Python scripts out of the Jenkinsfile

What went wrong: Groovy parses the entire Jenkinsfile before the shell runs it, and even inside `sh '''...'''` blocks, backslashes in regex patterns like `[\w.]+\.` trip the Groovy compiler. The cleanest fix — and also better engineering practice — is to put the Python logic in separate script files and call them from Jenkins.

Step A — Create the scripts folder

In the JupyterLab terminal:

```
bash
```

```
cd /opt/data/comp40771_lab9/lab9-advanced
mkdir -p scripts
```

Step B — Create `scripts/generate_report.py`

python

```
import xml.etree.ElementTree as ET
import os, json, re

results_dir = "results"
report_lines = []
summary = {
    "pipeline":      "lab9-advanced",
    "job":           os.environ.get("JOB_NAME",
"lab9-advanced"),
    "build":         os.environ.get("BUILD_NUMBER", "0"),
    "host":          os.environ.get("JENKINS_URL",
"http://jenkins-comp40771:8080"),
    "files_checked": [],
    "overall":       "SUCCESS",
}

xml_path = os.path.join(results_dir, "test_all.xml")

if os.path.exists(xml_path):
    tree = ET.parse(xml_path)
    root = tree.getroot()

    for suite in root.iter("testsuite"):
        total    = int(suite.attrib.get("tests",    0))
        failed   = int(suite.attrib.get("failures", 0))
        errors   = int(suite.attrib.get("errors",   0))
        skipped  = int(suite.attrib.get("skipped",  0))
        passed   = total - failed - errors - skipped
        report_lines += [
            f"Tests run : {total}",
```

```

        f"Passed      : {passed}",
        f"Failed      : {failed}",
        f"Errors       : {errors}",
        "",
    ]

    # Determine per-file result
    src_results = {}
    for case in root.iter("testcase"):
        for tag in ("failure", "error"):
            elem = case.find(tag)
            if elem is not None and elem.text:
                for line in elem.text.splitlines():
                    if "src/" in line and ".py" in line:
                        m = re.search(r"src/([\w.]+\.\py)", line)
                        if m:
                            fname = m.group(1)
                            src_results.setdefault(fname,
[]).append("FAIL")

    tracked = [
        "original_art11_sim.py",
        "improved_deepcoder_1_5b.py",
        "improved_qwen2_5_coder_1_5b.py",
    ]
    for fname in tracked:
        if fname in src_results:
            result = "FAIL"
            if fname != "original_art11_sim.py":
                summary["overall"] = "UNSTABLE"
            else:
                summary["overall"] = "FAILURE"
        else:
            result = "PASS"
        report_lines.append(f"{fname}: {result}")

```

```

        summary["files_checked"].append(fname)
else:
    report_lines.append("No XML results found.")
    summary["overall"] = "FAILURE"

summary["report_excerpt"] = " | ".join(report_lines[:6])

os.makedirs(results_dir, exist_ok=True)
with open(os.path.join(results_dir, "pipeline_report.txt"), "w")
as f:
    f.write("\n".join(report_lines))
with open(os.path.join(results_dir, "summary.json"), "w") as f:
    json.dump(summary, f, indent=2)

print("\n".join(report_lines))
print("\nSummary written to results/summary.json")

```

Step C — Create `scripts/publish_mqtt.py`

python

```

import json, os, time
import paho.mqtt.client as mqtt

with open("results/summary.json") as f:
    summary = json.load(f)

BROKER = os.environ.get("MQTT_BROKER", "mqtt-broker")
PORT   = int(os.environ.get("MQTT_PORT", "1883"))
TOPIC  = os.environ.get("MQTT_TOPIC",
"jenkins/pipeline/results")

connected = []

```

```

def on_connect(client, userdata, flags, reason_code,
properties):
    connected.append(True)

client =
mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION
2)
client.on_connect = on_connect
client.connect(BROKER, PORT, 60)
client.loop_start()

deadline = time.time() + 10
while not connected and time.time() < deadline:
    time.sleep(0.1)

client.publish(TOPIC, json.dumps(summary), qos=1, retain=False)
time.sleep(1)
client.loop_stop()
client.disconnect()
print(f"Published to {TOPIC}:")
print(json.dumps(summary, indent=2))

```

Step D — Replace the Jenkinsfile entirely

Overwrite `lab9-advanced/Jenkinsfile` with this clean version — no Python inline, no escaping issues:

```

groovy

pipeline {
    agent any

    environment {
        MQTT_BROKER = "mqtt-broker"
    }
}

```

```

    MQTT_PORT    = "1883"
    MQTT_TOPIC   = "jenkins/pipeline/results"
}

stages {

    stage('Checkout') {
        steps {
            checkout scm
        }
    }

    stage('Install Dependencies') {
        steps {
            sh 'pip install --quiet --no-cache-dir -r
requirements.txt'
        }
    }

    stage('Test All Files') {
        steps {
            sh 'mkdir -p results'
            sh '''
                python -m pytest tests/test_art11_sim.py \
                    --junit-xml=results/test_all.xml \
                    -v || true
            '''
        }
    }

    stage('Generate Report') {
        steps {
            sh 'python scripts/generate_report.py'
        }
    }
}

```

```

    stage('Publish to MQTT') {
        steps {
            sh 'python scripts/publish_mqtt.py'
        }
    }
}

post {
    always {
        junit allowEmptyResults: true, testResults:
'results/*.xml'
        archiveArtifacts artifacts: 'results/**',
allowEmptyArchive: true
    }
    success { echo 'Pipeline completed successfully.' }
    unstable { echo 'Pipeline completed - LLM variant
failures noted.' }
    failure { echo 'Pipeline failed.' }
}
}

```

Step E — Commit and push everything

bash

```

cd /opt/data/comp40771_lab9/lab9-advanced
git add Jenkinsfile scripts/generate_report.py
scripts/publish_mqtt.py
git commit -m "Part 4: fix Jenkinsfile - move Python scripts to
scripts/"
git push
http://N1419979:f74900a5622a5b2440fbb4e2dc7cae86aa3199d5@gitea:3
000/N1419979/lab9-advanced.git main

```

Then go to <http://localhost:8080/job/lab9-advanced> and click Build Now again.

Paste the Console Output if it fails.

It failed:

Console

[Download](#)[Copy](#)[View as plain text](#)

Started by user [N1419979](#)

Obtained Jenkinsfile from git <http://gitea:3000/N1419979/lab9-advanced.git>

[Pipeline] Start of Pipeline

[Pipeline] node

Running on [Jenkins](#) in /var/jenkins_home/workspace/lab9-advanced

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Declarative: Checkout SCM)

[Pipeline] checkout

The recommended git tool is: NONE

using credential lab9

Cloning the remote Git repository

Cloning repository <http://gitea:3000/N1419979/lab9-advanced.git>

> git init /var/jenkins_home/workspace/lab9-advanced # timeout=10

Fetching upstream changes from <http://gitea:3000/N1419979/lab9-advanced.git>

> git --version # timeout=10

> git --version # 'git version 2.47.3'

using GIT_ASKPASS to set credentials

> git fetch --tags --force --progress -- <http://gitea:3000/N1419979/lab9-advanced.git> +refs/heads/*:refs/remotes/origin/* # timeout=10

> git config remote.origin.url <http://gitea:3000/N1419979/lab9-advanced.git> # timeout=10

> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10

Avoid second fetch

> git rev-parse refs/remotes/origin/main^{commit} # timeout=10

Checking out Revision 0e0beca7b190a62943b6f11ce11e21cc9a185e9b

(refs/remotes/origin/main)

> git config core.sparsecheckout # timeout=10

```

> git checkout -f 0e0beca7b190a62943b6f11ce11e21cc9a185e9b # timeout=10
Commit message: "Part 4: fix Jenkinsfile - move Python scripts to scripts/"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] checkout
The recommended git tool is: NONE
using credential lab9
> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/lab9-advanced/.git #
timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url http://gitea:3000/N1419979/lab9-advanced.git #
timeout=10
Fetching upstream changes from http://gitea:3000/N1419979/lab9-advanced.git
> git --version # timeout=10
> git --version # 'git version 2.47.3'
using GIT_ASKPASS to set credentials
> git fetch --tags --force --progress -- http://gitea:3000/N1419979/lab9-advanced.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 0e0beca7b190a62943b6f11ce11e21cc9a185e9b
(refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 0e0beca7b190a62943b6f11ce11e21cc9a185e9b # timeout=10
Commit message: "Part 4: fix Jenkinsfile - move Python scripts to scripts/"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Install Dependencies)
[Pipeline] sh
+ pip install --quiet --no-cache-dir -r requirements.txt
ERROR: Could not find a version that satisfies the requirement pytest-junit (from
versions: none)

```

```
ERROR: No matching distribution found for pytest-junit
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test All Files)
Stage "Test All Files" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Generate Report)
Stage "Generate Report" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Publish to MQTT)
Stage "Publish to MQTT" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] junit
Recording test results
No test report files were found. Configuration error?
[Checks API] No suitable checks publisher found.
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] echo
Pipeline failed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
```

[Pipeline] End of Pipeline

ERROR: script returned exit code 1

Finished: FAILURE

Part 4 Fix — One-line problem in `requirements.txt`

What went wrong: `pytest-junit` does not exist on PyPI. The `--junit-xml` flag is built directly into `pytest` itself — no extra package needed.

Fix — Update `requirements.txt`

Open `lab9-advanced/requirements.txt` and replace it with just these three lines:

```
text
```

```
pytest
```

```
paho-mqtt>=2.0.0
```

```
requests
```

Then commit and push:

```
bash
```

```
cd /opt/data/comp40771_lab9/lab9-advanced
```

```
git add requirements.txt
```

```
git commit -m "Part 4: fix requirements.txt - remove  
non-existent pytest-junit"
```

```
git push
```

```
http://N1419979:f74900a5622a5b2440fbb4e2dc7cae86aa3199d5@gitea:3  
000/N1419979/lab9-advanced.git main
```

Then go to <http://localhost:8080/job/lab9-advanced> and click Build Now. This should be the only blocker — everything else in the pipeline was skipped only because pip failed first.